本章路線

這章要回答的核心問題是：在什麼 workload 與保證條件下，一個排程政策能把有限 CPU 配得正確、可預測且有效率？

前置知識。 process 狀態與佇列、context switch、機率與期望值。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

學習目標

- 推導等待、周轉與回應時間
- 比較 FCFS、SJF、SRTF、RR、priority 與多級回饋佇列
- 檢查搶占、飢餓、優先權倒置與即時可排程性

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

6.1 基本概念

- process 在 CPU execution 與 I/O wait 間交替。CPU burst 是一段不中斷的執行需求，I/O burst 是等待外部完成的區間。
- short-term scheduler 由 ready queue 選出 kernel thread；dispatcher 完成 context switch、模式切換與跳回目標程式。dispatch latency 是這段無用工作的時間。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

6.1.1 排程器的三種時間尺度

- long-term scheduler 控制哪些工作可進入系統，影響多程式度；short-term scheduler 只選下一個執行者；medium-term scheduler 可把工作換出以降低記憶體壓力。
- 批次系統的 admission 決定可改變後續 workload；互動系統更在意回應時間，而非只追求 CPU 忙碌。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

6.1.2　搶占與非搶占

- nonpreemptive 排程只在 running 完成、自願讓出或阻塞時切換；preemptive 可在時間片到期或更高優先權者到來時切換。
- 搶占改善回應與公平，但每次切換都要保存狀態、維護 run queue，並防止共享核心資料被交錯破壞。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

6.1　模型、時序與反例：基本概念

模型假設。 每個工作有可描述 arrival、CPU burst 與 I/O burst；排程器可保存/恢復執行上下文，queue 狀態轉移具原子性。

不變量與時序。 running 任務每 CPU 至多一個，ready/blocked queue 不重複入列；任意時間表滿足資源釋放與 arrival 順序。

反例或邊界。 忽略 context switch 成本時，極短 quantum 看似公平，實際可能大部分 CPU 都在切換。

完整算例。 P1 burst=5後等待I/O，P2 burst=2；單CPU、switch cost=1，FCFS為P1[0,5)、switch[5,6)、P2[6,8)。有用工作占比是7/8=87.5%，切換開銷占1/8=12.5%。CPU整段都在執行工作或核心切換，沒有idle，因此若CPU utilization定義為busy/elapsed，仍是8/8=100%。有用工作效率與CPU忙碌率不是同一個量；省略switch會把總時間錯算為7。

思考題。 哪條時間線事件能同時破壞 queue 不重複與工作完成順序？

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

6.2 排程準則

- CPU utilization 是 CPU 忙碌時間占觀察區間的比例；throughput 是單位時間完成的工作數。這兩個效率指標可能提高，但個別工作的 waiting、response 或 turnaround 仍變差。
- waiting time 是任務在 ready state 的累積時間，不含實際 CPU service 與 blocked I/O。turnaround time 從 arrival 到 completion。排程課本把 response time 定為 arrival 到第一次取得 CPU；若服務契約要看首次可見輸出，必須另量 output time。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

6.2.1 唯一公式與首輸出語義

- 令 task i 的 arrival 為 a_i、ready intervals 的總和為 W_i、completion 為 f_i。turnaround_i=f_i-a_i，waiting_i=W_i。blocked time 既不屬 CPU service，也不屬 ready waiting。
- 排程 response_i=s_i-a_i，其中 s_i 是第一次取得 CPU 的時刻。若產品指標定義為首次可見輸出 o_i，則 user-visible response_i=o_i-a_i。兩個 response 都可能有價值，但名稱與量測點必須明示。
- 固定例 a=2、s=5、o=9、f=14，且 2 至 5 位於 ready queue：waiting=3、排程 response=3、首輸出 response=7、turnaround=12。waiting 不可改寫成 turnaround 減 burst 而忽略 blocked 時段。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

6.2　模型、時序與反例：排程準則

模型假設。 時間單位一致；arrival、first CPU、first visible output、completion 均有可觀測事件；ready 與 blocked intervals 分開記錄。

不變量與時序。 waiting_i 只累加 ready intervals；turnaround_i=completion_i-arrival_i；排程 response 與首輸出 response 各自使用唯一指定的事件。

反例或邊界。 a=2、s=5、o=9、f=14 的工作若把 blocked 等待算入 waiting，會得到錯誤的 waiting=12，把周轉時間重新命名成 waiting。

完整算例。 上述固定例得到 waiting=3、排程 response=3、首輸出 response=7、turnaround=12；四個數值回答不同問題，且 waiting/response 不由 completion 再重複定義。

思考題。 互動服務若回報 response=0，必須同時揭露它量的是第一次 CPU 還是首次使用者可見輸出。

6.2　waiting、first CPU 與 first visible output

本節只保留一組定義。令 task i 的 arrival 為 a_i，ready state intervals 的總和為 W_i，第一次取得 CPU 的時間為 s_i，第一次產生使用者可見輸出的時間為 o_i，完成時間為 f_i。

$$\text{waiting}_i = W_i, \quad \text{turnaround}_i = f_i - a_i$$

排程課本的 response time 使用第一次取得 CPU：

$$\text{schedule response}_i = s_i - a_i$$

若介面契約量的是首次可見輸出，必須另記：

$$\text{visible response}_i = o_i - a_i$$

固定案例的 arrival $a = 2$、第一次取得 CPU $s = 5$、首次輸出 $o = 9$、完成 $f = 14$，且 2 至 5 都在 ready queue。結果是 waiting=3、schedule response=3、visible response=7、turnaround=12。這些值回答四個不同問題，不能相加。

6.2 blocked time 為何不能算 waiting

若同一任務在完成前另有 4 個時間單位等待 I/O，則 blocked time 增加完成時間，但 ready waiting 不增加。等待 I/O 的 task 不在 ready queue，不能與 CPU dispatch 的 waiting 混算。

設task A arrival0，完整時間線為ready[0,2)、CPU[2,4)、blocked[4,8)、ready[8,11)、CPU[11,12)。完成時間f=12，turnaround=12−0=12，ready waiting=2+3=5，CPU service=2+1=3，blocked=4，且5+3+4=12。若把blocked4也算waiting會誤得9，與ready waiting的定義不符。

平均 waiting 相同也不代表最大 response 相同。工作 A 與 B 都等待 4 units，但 A 的第一次 CPU 在第 4 unit、B 在第 8 unit；若 B 另外要等到輸出才可見，visible response 還會更晚。報告排程品質時，至少要同時標出定義、量測事件與 percentile。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

6.3 排程演算法

- 演算法需明確比較鍵、搶占時機、tie-break 與 queue 管理。平均等待較低不代表每個工作都有上限。
- 對同時可用、單一 CPU burst、已知長度且忽略切換成本的一組工作，非搶占 SJF 最小化平均等待；SRTF 是它的可搶占版本。實務無法完美預知未來 burst，只能用歷史估計。
- SJF 的交換論證：若同時可用的相鄰兩工作長度 x>y，先 x 後 y 對兩者等待和貢獻為 x，交換後為 y，其餘工作等待不變，故每次消去逆序都不增加等待總和。不同 arrival、允許刻意 idle 或有估計誤差時不能直接沿用此最適性結論。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

6.3.1 FCFS 與 convoy effect

- FCFS 依 arrival 順序執行，實作簡單且無搶占。長工作先到時，後方短工作一起等待，形成 convoy effect。
- 例：P1 burst 120、P2/P3 各 burst 10，arrival 都為 0，FCFS 等待為 0、120、130，平均 83.33；若短工作先做，等待分布完全不同。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

6.3.2 SJF 與 SRTF

- 非搶占 SJF 在 CPU 空閒時選最短 burst；SRTF 在更短工作抵達時搶占。兩者都以已知 burst 為前提。
- 長工作有飢餓風險：源源不絕的短工作會使長工作延後。tie-break 影響具體時間表但不等於公平保證。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

6.3.3 Round Robin

- RR 以固定 time quantum 輪流執行。所有工作 burst 不超過 q 時近似 FCFS；q 太小則切換成本占比上升。
- 例：P1/P2/P3 arrival 0、burst 24/3/3、q=4，時間線為 P1[0,4)、P2[4,7)、P3[7,10)、P1[10,14)、P1[14,18)、P1[18,22)、P1[22,26)、P1[26,30)，三份 burst 總和為 30。後段只有 P1 可執行時，時間片邊界不必造成跨 thread context switch；timer／排程檢查與實際切換需分開計算。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

6.3.4 Priority 與多級佇列

- priority 排程可選最高優先權；低優先權工作可能 starve。aging 隨等待時間提高有效 priority，避免無限延後。
- multilevel queue 區分 foreground/background，multilevel feedback queue 允許工作依行為換級；時間片與升級規則本身就是 policy。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

6.3　模型、時序與反例：排程演算法

模型假設。 每種演算法要說明 burst 已知或估計、arrival 可同時、preemption 與 tie-break 規則。

不變量與時序。 CPU 時間表不重疊且每段長度不超過該工作剩餘 burst；每次 arrival 與完成都被更新。

反例或邊界。 SJF 在 burst 不斷被低估的 workload 中可能讓長工作反覆被延後。

完整算例。 P1/P2/P3 burst=8/4/2、arrival=0。FCFS wait=0、8、12，平均 20/3≈6.67；SJF order 2、4、8，wait=0、2、6，平均 8/3≈2.67。RR q=2 的 completion依P1/P2/P3為14/10/6，三份 burst 的總 CPU 時間仍是14。

思考題。 如果 burst 只是估計值，哪個演算法保證需要被改寫？

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

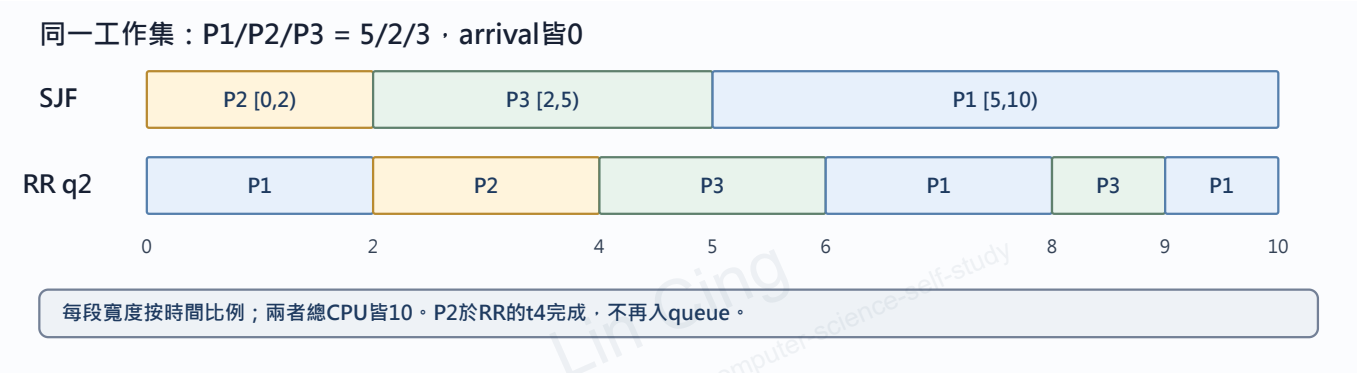
6.3　SJF與RR：同一個5/2/3工作集

固定P1/P2/P3皆arrival0，CPU bursts5/2/3，切換成本0。非搶占SJF依序P2[0,2)、P3[2,5)、P1[5,10)，completion依P1/P2/P3為10/2/5，waiting為5/0/2。

RR q2的ready queue初序P1,P2,P3；依序為P1[0,2)、P2[2,4)、P3[4,6)、P1[6,8)、P3[8,9)、P1[9,10)，共6段。P2在t4已完成，不可再次入列；completion為10/4/9，waiting為5/2/6，first response為0/2/4。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

6.3 SJF與RR：同一個5/2/3工作集 （續）



兩者useful CPU總量都是10，但SJF平均waiting=7/3，RR為13/3；response分布亦不同。前段8/4/2的另一個固定案例則有RR q2的P1[0,2)、P2[2,4)、P3[4,6)、P1[6,8)、P2[8,10)、P1[10,12)、P1[12,14)，completion為14/10/6。不得把不同輸入的completion混到同一個表。

6.4 Thread scheduling

- user-level thread 由 thread library 在 process-contention scope 內競爭；kernel thread 由 kernel 在 system-contention scope 中競爭 CPU。
- one-to-one 模型通常讓每個 user thread 對應可被核心排程的 thread；many-to-one 的阻塞 system call 可能卡住整個 process。

6.4　模型、時序與反例：Thread scheduling

模型假設。 thread library 與 kernel 的映射已知；阻塞 system call 是否釋放 kernel execution entity 已明確。

不變量與時序。 同一 kernel execution entity 不能同時執行兩個 context；user thread 阻塞不應在映射允許時阻塞無關 thread。

反例或邊界。 many-to-one 中一個阻塞 I/O 可讓 process 內其他 runnable threads 全停。

完整算例。 兩個 user threads：T1 等 network、T2 計算 4 units。one-to-one 下 T1 blocked 時 T2 可在時間0開始並在4完成；many-to-one 只有1 kernel execution entity，若 T1 的 blocking call 不讓出，T2 的開始時間未知，不能保證在4完成。

思考題。 thread API 的並行承諾在何種映射與阻塞模型下成立？

6.5　多處理器排程

- processor affinity 可保留快取與 TLB 的溫暖狀態，但過度綁定會造成負載不均；load balancing 與 locality 形成取捨。
- SMP 需避免同一 runnable thread 被兩個 CPU 同時執行。run queue 設計、migration 與 stealing 都需原子同步。

6.5　模型、時序與反例：多處理器排程

模型假設。 多核心共享或分區 run queue；migration 有鎖定與 memory/cache 一致性成本。

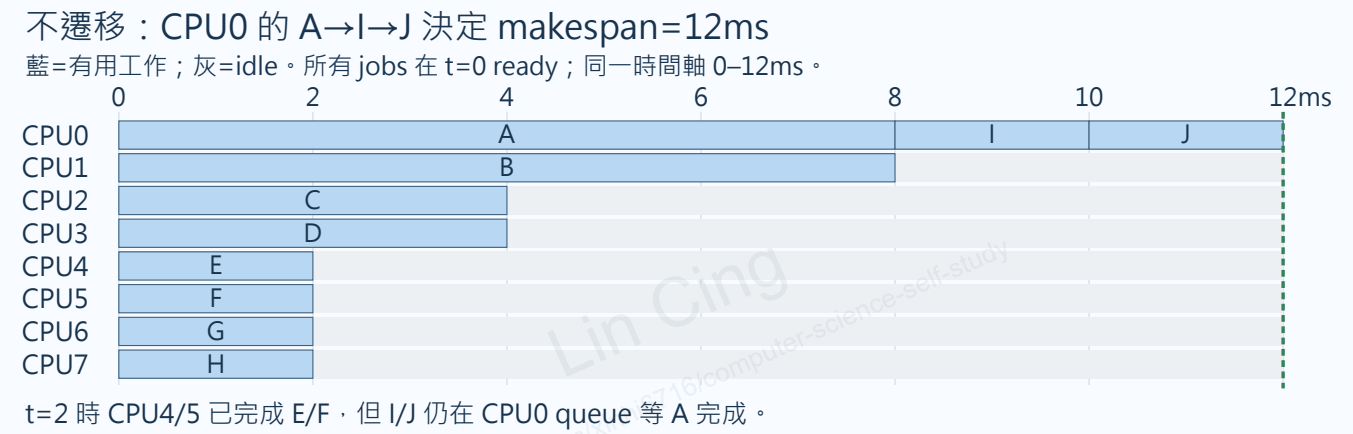
不變量與時序。 同一 runnable thread 不得同時被兩個 processors 執行；負載平衡後仍保留必要 affinity。

反例或邊界。 全體任務綁在一個核心以保快取，雖然 data locality 好，其他核心卻閒置。

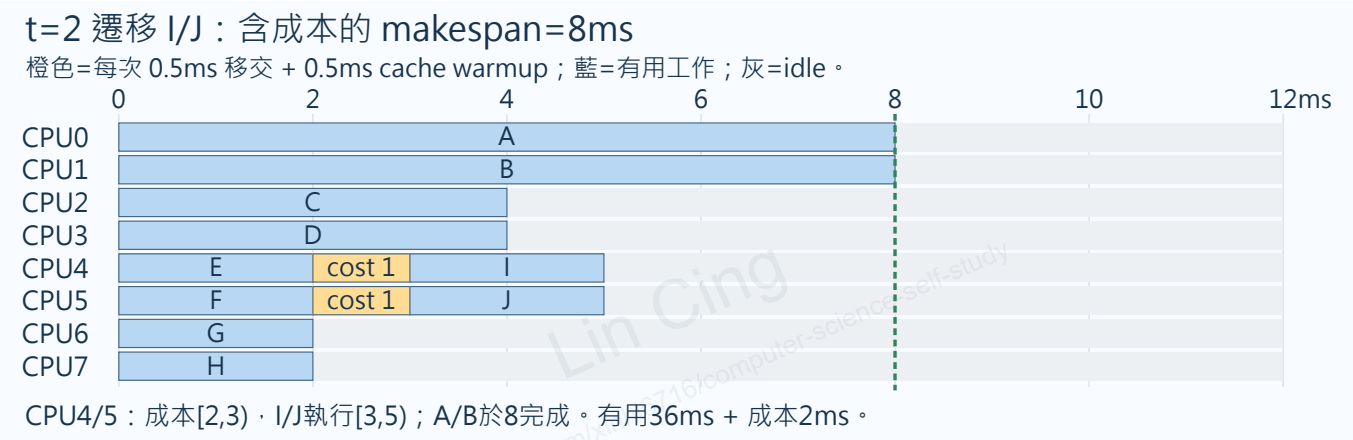
完整算例。 固定8cores、所有jobs在t=0就緒、不可切分、單job不得同時跑在兩core。Jobs A/B各8ms、C/D各4ms、E/F/G/H/I/J各2ms，共36ms有用工作。初始CPU0 queue=[A,I,J]；CPU1至7分別只有B、C、D、E、F、G、H。

不遷移時CPU0依序A[0,8)、I[8,10)、J[10,12)，makespan=12ms。t=2時CPU4/5已做完E/F；在queue lock下各移除尚未執行的I/J，移交其唯一owner。每次遷移明定0.5ms queue/transfer加0.5ms cache warmup，故兩core各在[2,3)付成本、[3,5)執行I/J。CPU0只執行A至8，makespan降為8ms，speedup=12/8=1.5；有用工作仍36ms，新增成本共2ms。這是給定成本的教學模型，不是實機測量。

6.5　模型、時序與反例：多處理器排程 （續）



6.5　模型、時序與反例：多處理器排程 （續）



單job A長8ms給下界T≥8，所以本例達到此下界。若每次遷移成本改為c，makespan=max(8,2+c+2)=max(8,4+c)；c<8才比12ms嚴格改善。資料相依、硬性affinity或不同cache成本會改變模型，不能只因core閒置就宣稱遷移一定有益。

6.6　即時排程：先分清 task、job 與期限

週期工作（task）τ_i 反覆產生一筆筆待執行的 job。以下統一使用執行時間上界 C_i、週期 T_i、相對期限 D_i。從時刻 0 同步釋放時，第 k 筆 job（k 從 0 起）的釋放時刻與絕對期限為

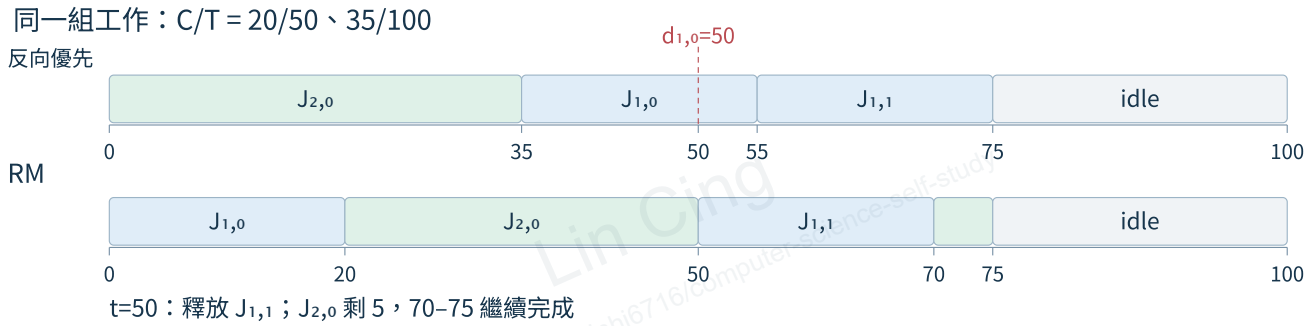
$$r_{i,k} = kT_i, \qquad d_{i,k} = r_{i,k} + D_i.$$

本節基本模型：單 CPU、可立即搶占、工作彼此獨立，沒有鎖阻塞、I/O、自我暫停、切換或中斷成本；除末頁外 $D_i=T_i$。 到達時只把新 job 放進 ready set，未必換人執行。完成恰在期限上算成功；時刻 t 先計入 [前一事件,t) 的執行，再處理 t 的到達。

量	問的問題	不能混用的量
C_i	每筆最多需要多少 CPU 時間？	不是 response time
T_i	同一 task 隔多久產生下一筆？	不是當前 job 剩餘期限
D_i	從釋放起多久必須完成？	絕對期限為 $r_{i,k}+D_i$
$U=\Sigma C_i/T_i$	長期 CPU 需求占比是多少？	不單獨描述短時間擁塞

RM：週期短的 task 優先，不是剩餘工作少者優先

Rate-monotonic（RM）把**較短週期設為較高固定優先權**；新來的高優先權 job 可搶占。令 τ_1=(C_1=20,T_1=D_1=50)、τ_2=(35,100,100)，時間單位均為 ms。



若把 τ_2 排在前面，$J_{1,0}$ 只能在 [35,55) 執行，超過期限 50。RM 則先做 $J_{1,0}$；$J_{2,0}$ 在 [20,50) 已做 30，t=50 被 $J_{1,1}$ 搶占，**剩下的 5 不會重做**，於 [70,75) 完成。

RM 的完成時刻為 $J_{1,0}$=20、$J_{1,1}$=70、$J_{2,0}$=75，對應期限 50、100、100。t=100 是共同週期邊界，舊 job 全已完成；同一 ready 狀態隨後重現，故整個排程可每 100 ms 重複。

RM 利用率界：通過可保證，沒通過要再分析

在本節獨立、可搶占、D_i=T_i 的理想單 CPU 模型，n 個週期 task 的經典**充分條件**為

$$U = \sum_i C_i/T_i \le n(2^{1/n} - 1).$$

n=2 時界為 $2(\sqrt{2}-1)\approx 0.8284$；前例 U=20/50+35/100=0.75，通過。$n\to\infty$ 時界趨近 $\ln 2\approx 0.6931$。這些數字是對所有符合模型工作集的保證界，**不是每個 RM 工作集最多只能用這麼多 CPU**。

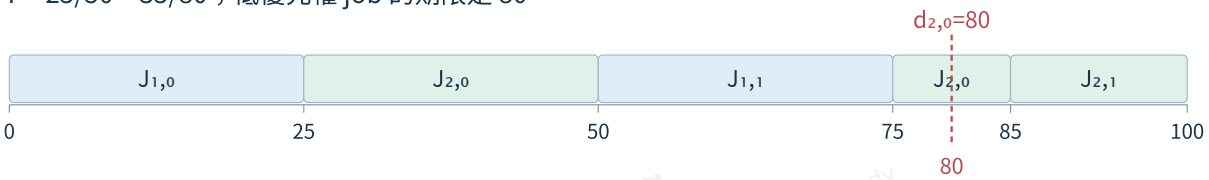
反例：τ_1=(C=1,T=D=2)、τ_2=(2,4,4)，U=1。RM 在 [0,1) 做 $J_{1,0}$，[1,2) 做 $J_{2,0}$，[2,3) 做 $J_{1,1}$，[3,4) 做 $J_{2,0}$ 的剩餘 1。三筆完成於 1、3、4，期限分別為 2、4、4。t=4 無積欠且同時再釋放，故可重複，RM 仍可排程。

判斷順序：先核對假設，再用充分測試；不通過時，以更精確的干擾分析或可證明重複的時間表判定。單憑 U>0.8284 不能宣告失敗；U≤1 也不能保證任意固定優先權成功。

同樣不到滿載，RM 仍會讓某筆 job 超時

改成 τ_1=(25,50,50)、τ_2=(35,80,80)。U=25/50+35/80=15/16=0.9375；RM 仍讓 τ_1 優先。

RM：C/T = 25/50、35/80；低優先權 job 的期限是 80



t=80：舊 $J_{2,0}$ 還剩 5；新 $J_{2,1}$ 另行加入

$J_{2,0}$ 在 [25,50) 做了 25，剩 10；[50,75) 被 $J_{1,1}$ 搶占；[75,80) 只能再做 5，**到了期限 80 仍剩 5**，所以此時已失敗。若超時 job 不丟棄，續做到 85 才完成。t=80 釋放的 $J_{2,1}$ 與舊 $J_{2,0}$ 是兩筆不同 job，圖中沒有把它們合併。

U<1 只表示長期總量未超過 CPU 供給；固定優先權仍可能把某個短區間擠滿，讓低優先權 job 錯過期限。證明失敗的是 t=80 的剩餘需求，不是「超過 RM 充分界」。

用干擾次數核對 RM 的完成時刻

只分析兩個同步釋放的 task，且 τ_1 優先、$C_1 \le T_1$、沒有阻塞。$J_{2,0}$ 在完成前長度 R 的區間 [0,R) 內，τ_1 於 $0, T_1, 2T_1, \cdots$ 釋放；共有 $\lceil R/T_1 \rceil$ 筆會干擾。**在 R 才釋放的新 job 不算入先前區間**。

$$w^{(0)} = C_2, \qquad w^{(k+1)} = C_2 + \left\lceil w^{(k)}/T_1 \right\rceil C_1.$$

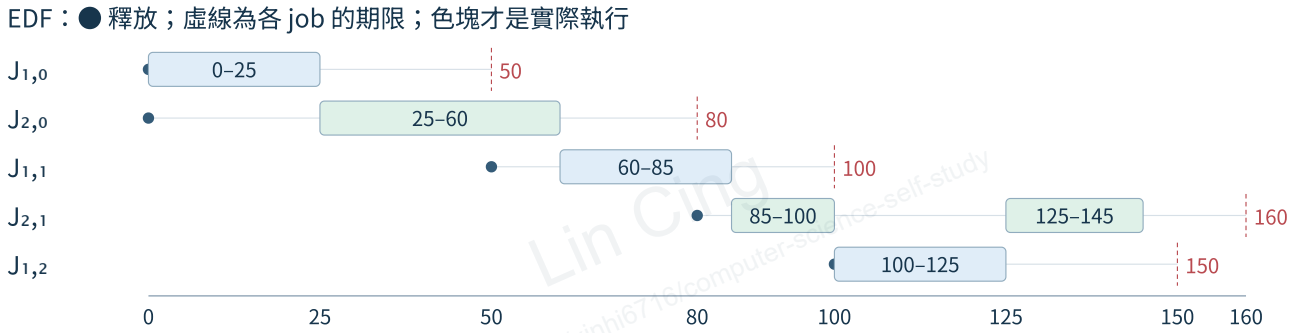
從 C_2 往上代入，求出這筆同步 job 的最早固定點。若遞增值已超過 D_2，期限內不可能完成；固定點 $\le D_2$ 才通過此筆檢查。

工作集	$w^{(0)}$	$w^{(1)}$	$w^{(2)}$	$w^{(3)}$	核對結果
(20,50)、(35,100)	35	55	75	75	75≤100，對上圖
(25,50)、(35,80)	35	60	85	85	85>80，已超時
(1,2)、(2,4)	2	3	4	4	4=4，恰在期限完成

迭代不減：右側對 w 單調，且首步不小於 C_2。若存在期限內固定點 R^*，歸納得每一步 $w \le R^*$；因此迭代超出期限即可排除期限內固定點。這裡只推導同步兩 task 的首筆；任意相位、多 task 或有 blocking 的完整最壞回應時間分析，還需要相應的 critical-instant 與干擾模型，不能由這張表直接推出。

EDF：比較的是每筆 job 的絕對期限

Earliest-deadline-first（EDF）每次從 ready jobs 選**絕對期限最早**者。沿用 τ_1=(25,50,50)、τ_2=(35,80,80)；平手時先維持 current，否則依 task 編號、job 編號。



t=50 新 $J_{1,1}$ 的期限 100 比 current $J_{2,0}$ 的 80 晚，故 **$J_{2,0}$ 續作至 60 完成**。t=100 新 $J_{1,2}$ 的期限 150 比 current $J_{2,1}$ 的 160 早，才搶占。

圖只展開前 150 ms；有限前綴通過不代表未來全通過，後面的 EDF 定理才給一般保證。被搶占者保留剩餘量，不重新從 C_i 開始。

EDF 每一次選擇，都能回到 ready set 核對

表中列事件處理後、下一段執行前的 ready jobs；括號依序為「剩餘 CPU，絕對期限」。只列尚未完成的 job。

t (ms)	ready set	下一段執行	選擇理由
0	$J_{1,0}$(25,50)、$J_{2,0}$(35,80)	$J_{1,0}$：0–25	50<80
25	$J_{2,0}$(35,80)	$J_{2,0}$：25–50	唯一 ready
50	$J_{1,1}$(25,100)、$J_{2,0}$(10,80)	$J_{2,0}$：50–60	80<100，續作
60	$J_{1,1}$(25,100)	$J_{1,1}$：60–80	唯一 ready
80	$J_{1,1}$(5,100)、$J_{2,1}$(35,160)	$J_{1,1}$：80–85	100<160，續作

前五個事件的重點：$J_{2,0}$ 在 50 面對新到達仍續作；$J_{1,1}$ 在 80 也續作。到達只改 ready set，只有 current 不再是最早到期者時才搶占。

EDF 在 100 搶占：同一規則為何得到不同選擇？

仍以（剩餘 CPU，絕對期限）列 ready jobs。

t (ms)	ready set	下一段執行	選擇理由
85	$J_{2,1}$(35,160)	$J_{2,1}$：85–100	唯一 ready
100	$J_{1,2}$(25,150)、$J_{2,1}$(20,160)	$J_{1,2}$：100–125	150<160，搶占
125	$J_{2,1}$(20,160)	$J_{2,1}$：125–145	唯一 ready
145	空	idle：145–150	沒有可執行 job

J_1 的前三筆完成於 25、85、125，期限 50、100、150；J_2 的前兩筆完成於 60、145，期限 80、160。**job 到達、優先權重排、實際切換是三件事**；50 與 80 都有到達，但 current 仍是最佳選擇。

為什麼理想 EDF 在 D=T 時只需 U≤1？

考慮前述單 CPU 模型，自 t=0 無積欠開始，允許不同 task 有不同非負釋放相位。若 U>1，長度 t 內到期的需求為 Ut−O(1)，t 夠大就超過 t，故任何排程都不可能永遠準時。

反過來，假設 U≤1，EDF 卻首次在時刻 b 錯過期限。只看絕對期限 ≤b 的 jobs；向前找它們**持續有未完成工作**的最後一段 [a,b)，長度 L=b−a。在 a 前沒有這類積欠，故區間內要完成的這類 jobs 都在 a 或之後釋放。EDF 在整段只執行這類工作。

因 $D_i=T_i$，同一 task 在 [a,b] 內「釋放且到期」的 jobs 最多 $\lfloor L/T_i \rfloor$ 筆：首筆最快在 $a+T_i$ 到期，以後每次至少再隔 T_i。因此必須在這段處理的總需求滿足

$$\sum_i N_i C_i \le \sum_i \lfloor L/T_i \rfloor C_i \le L \sum_i C_i/T_i = LU \le L.$$

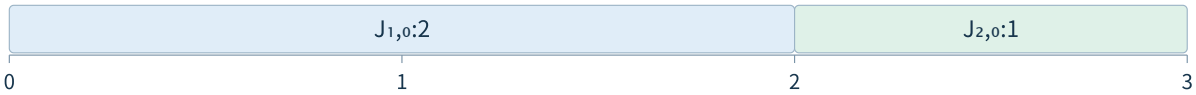
但首次超時表示 EDF 已連做 L 時間仍沒做完，需求必須 **>L**，矛盾。所以模型內 U≤1 充分；加上前段必要性，即為充要條件。這個證明靠的是每個區間的需求上界，並非「平均沒滿就一定沒事」。

前例 U=15/16，故 EDF 可保證所有 jobs；實機切換成本、鎖阻塞、多核心或 D<T 都會改變上述前提，不能照搬結論。

相對期限短於週期時，低利用率也可能無解

令 τ_1、τ_2 都是 C=2、T=10，但 D=3，同步在 0 釋放。U=0.4，然而兩筆 job 都須在 t=3 前完成，合計需要 4 ms；單 CPU 在 [0,3) 只供應 3 ms。**不論 RM、EDF 或其他政策，至少有一筆超時**。

C=2、T=10、D=3：兩筆都在 0 到達，3 之前需要 4



供給 3 < 需求 4；t=3 時 $J_{2,0}$ 還剩 1

對一般 $D_i \le T_i$，長度 L 內可能「釋放且到期」的最大需求為

$$\mathrm{dbf}_i(L) = \max\left(0, \left\lfloor \frac{L - D_i}{T_i} \right\rfloor + 1\right) C_i.$$

首筆需留 D_i 時間；若 $L \ge D_i$，此後每隔 T_i 可再放一筆，故得到上式。此例 L=3，每個 task 的 dbf=2，合計 4>3，已足以證明不可排程。只檢查某一個 L 通過不能證明全部區間都通過。

即時分析的第一步是把**需要多少 CPU、何時到達、最晚何時交付**分開寫。priority inheritance 可處理特定鎖造成的優先權反轉，但不會把這個 4>3 的容量矛盾消掉。

6.7　作業系統例子

- Solaris 依 class-specific priority 轉成 global priority；TS/IA class 使用 multilevel feedback，priority 0 的 quantum 為 200 ms、priority 59 為 20 ms，CPU 用盡與 sleep 返回分別套用不同新 priority。Windows 7 的三十二級 queue 與 boost 保留原書版本界線。
- Linux 3.2 CFS 的 red-black tree 只排 ready entity；running task 暫離 tree。vruntime 增量為實際執行時間×1024/weight，選取最小 vruntime；nice 0 的 weight 為 1024，nice 5 為 335。
- v3.2 的 ideal_runtime 為 period×weight/runnable total weight；runnable 數不超過 8 時 period=6 ms。一般週期 tick 只在 delta_exec>ideal_runtime 時搶占，queued HRTICK 到達 slice 即重排。
- 睡醒位置為 max(舊 vruntime,min_vruntime−3 ms)。喚醒 task 要在 virtual-time lead 嚴格大於 1 ms 才搶占；last/next/skip buddy 只能在公平差距允許時影響候選。

6.7　模型、時序與反例：作業系統例子

模型假設。 單 CPU；P1=8、P2=4 都在 t=0 runnable，P3=2 睡眠至 t=9；三者 nice=0、weight=1024。CFS 初始 vruntime 都正規化為 0，P1 先入 tree；GENTLE_FAIR_SLEEPERS=true、wakeup granularity=1 ms、CONFIG_SCHED_HRTICK=y；省略 timer jitter、interrupt、switch 與 cache 成本。

不變量與時序。 ready 集合不含 current；每次由 current 以外的 tree leftmost 選出最小 vruntime。n≤8 時 period=6 ms；兩 task 等權重時 ideal_runtime=3 ms。只有 HRTICK、期滿 wakeup 條件或 task completion 造成事件，不能憑圖硬插切換。

反例或邊界。 t=8 沒有 completion、tick 或 wakeup，所以 CFS 不會在 6–9 切成 6–8 與 8–9。另一情形的 1 ms 週期 tick 在 delta_exec=3 時不切換，因 v3.2 條件是嚴格大於；要等 delta_exec=4 才一般 tick 搶占。

完整算例。 HRTICK 軌跡：P1[0,3)、P2[3,6)、P1[6,9)、P3[9,11)、P2[11,12)、P1[12,14)。t=6 兩者 vruntime=3，先入 tree 的 P1 取左端；t=9 P1 已累積 6 ms，P3 以 min_vruntime=3、sleep credit=3 放回 vruntime=0，且 current−P3=6>1 ms，使 P3 立即搶占。完成時間 P1/P2/P3=14/12/11。

思考題。 關閉 HRTICK、保留 1 ms 週期 tick 並讓 P3 在 t=9 醒來時，第一個 3 ms slice 為何要到 t=4 才切換？

6.7　Linux 3.2 CFS 的事件模型

本頁只回答一個可核驗的問題：在固定的單 CPU 模型裡，v3.2 CFS 何時真正切換。P1、P2 的 burst 為 8、4 ms，P3 的 burst 為 2 ms；P1/P2 在 t=0 runnable，P3 睡眠至 t=9。三者 nice=0、weight=1024，CPU 切換、timer jitter 與 interrupt 成本均為 0。

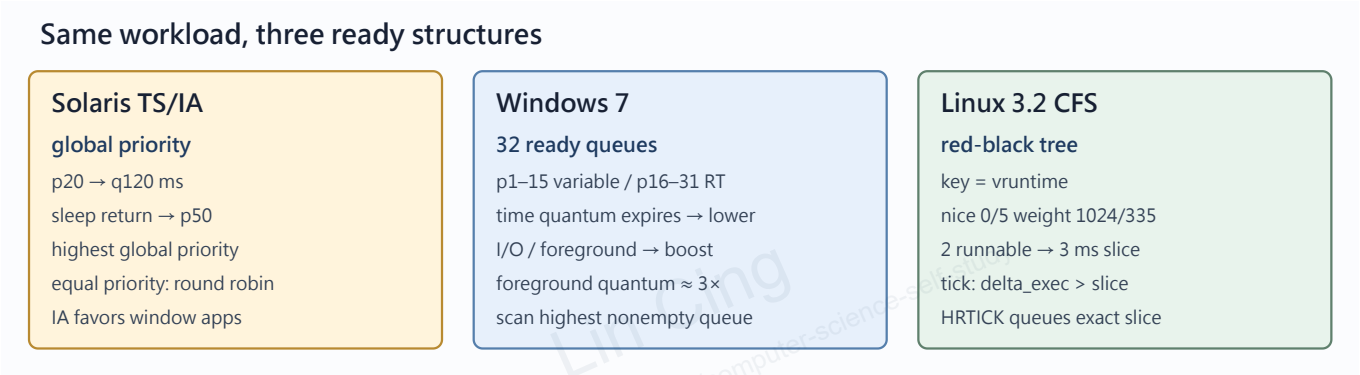
教材把 P1/P2 初始 vruntime 正規化為 0，P3 睡眠前的 vruntime 也是 0；同值時依 P1、P2 的入樹順序取左端。這不冒充實際開機時 START_DEBIT 的絕對 offset，因為三者使用同一個選取鍵位移時，相對次序不變。模型另固定 GENTLE_FAIR_SLEEPERS=true、wakeup granularity=1 ms 與 CONFIG_SCHED_HRTICK=y。

ideal_runtime 是可執行的 wall-time slice，不是固定的全域 quantum。令 n 為 runnable entity 數：

$$period = \begin{cases} 6\ \text{ms}, & n \le 8, \\ 0.75n\ \text{ms}, & n > 8, \end{cases} \qquad ideal_i = period \times \frac{weight_i}{\sum_j weight_j}.$$

本題 n=2，因此 period=6 ms、每個 nice=0 task 的 ideal_runtime=3 ms。current 從 ready red-black tree 移除；重新選取時，舊 current 只有仍為 runnable 才依更新後的 vruntime 回樹。已阻塞或完成的 task 不在 ready tree，阻塞者須等喚醒才重新入樹。排程器依相應選取規則取得下一個 current，選定後的 ready 欄不含它。

6.7 Linux 3.2 CFS 的事件模型 (續)



6.7 逐事件 CFS trace：t=8 沒有切換理由

下表的 ready 不含 current。min_vruntime 是當時 runqueue 的 virtual-time 基準；P3 睡眠時不在 ready tree。

時刻與事件	current	其他 ready	min vruntime	本次 ideal_runtime	選取理由
t=0 初始	P1	P2@0	0	3	P1/P2 同為 0；P1 先入樹
t=3 queued HRTICK	P1→P2	P1@3	0	3	P1 已跑滿 3 ms
t=6 queued HRTICK	P2→P1	P2@3	3	3	兩者同為 3；P1 是既有 leftmost
t=9 HRTICK 與 P3 wake	P1→P3	P2@3、P1@6	3	2	P3 放回 0；current 領先 6>1 ms
t=11 P3 完成	P3→P2	P1@6	3	3	P2 的 3 小於 P1 的 6；選定 P2 後將它移出 ready

6.7　逐事件 CFS trace：t=8 沒有切換理由 （續）

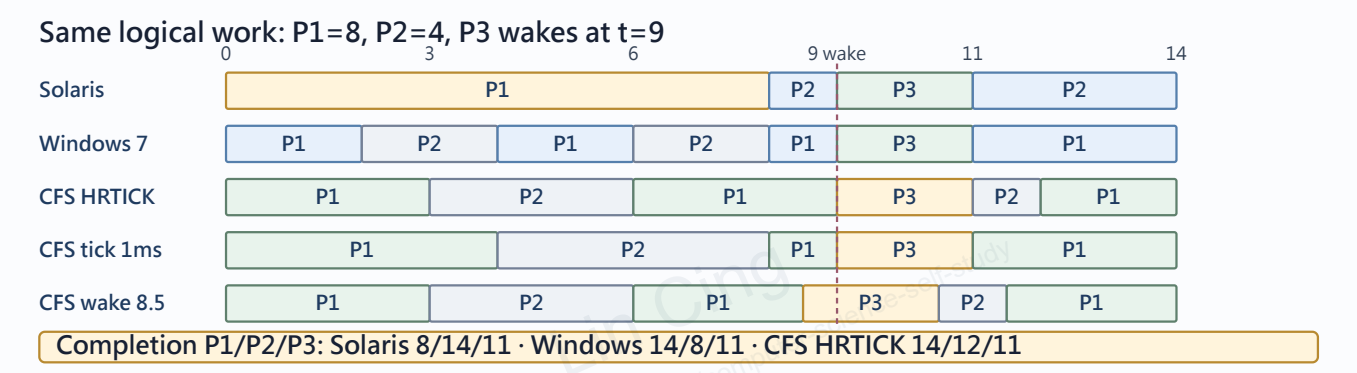
時刻與事件	current	其他 ready	min vruntime	本次 ideal_runtime	選取理由
t=12 P2 完成	P2→P1	無	6	6	只剩 current P1，剩餘 2 ms
t=14 P1 完成	P1→無	無	8	—	沒有 runnable task；保留最後 min_vruntime

關鍵事件是 t=9。P1 最後一次領到 CPU 是 t=6，ideal_runtime=3 ms，所以它在 6–9 可以一直跑到 HRTICK。P2 仍是 ready，min_vruntime=3 ms。P3 的放回值為

$$\max(v_{\mathrm{P3,sleep}}, min_vruntime - 3) = \max(0, 3 - 3) = 0.$$

current P1 在 t=9 的 vruntime=6，喚醒條件是 v_current−v_wakeup>1 ms；6−0=6，嚴格大於 1，所以 P3 立即取得 CPU。完成時間是 P1=14、P2=12、P3=11 ms，不是 14/9/11。

6.7　逐事件 CFS trace：t=8 沒有切換理由 （續）



6.7　tick、HRTICK 與 sleep placement 的敏感度

同一個 workload 在不同 timer 模式下不能假裝只有一張答案。

設定	Gantt	completion P1/P2/P3	機制差異
HRTICK，P3 wake t=9	P1[0,3)、P2[3,6)、P1[6,9)、P3[9,11)、P2[11,12)、P1[12,14)	14/12/11	queued HRTICK 在 slice 邊界直接重排
無 HRTICK，1 ms tick，P3 wake t=9	P1[0,4)、P2[4,8)、P1[8,9)、P3[9,11)、P1[11,14)	14/8/11	delta_exec=4>3 才週期 tick 搶占
HRTICK，P3 wake t=8.5	P1[0,3)、P2[3,6)、P1[6,8.5)、P3[8.5,10.5)、P2[10.5,11.5)、P1[11.5,14)	14/11.5/10.5	wakeup lead 5.5>1 ms，真正發生喚醒搶占

check_preempt_tick 的 delta_exec > ideal_runtime 是嚴格不等號。沒有 HRTICK 時，1 ms tick 在 t=3 只看到 3>3 為 false，故 P1 到 t=4 才被一般 tick 搶占。HRTICK 由另一個高解析度 timer 在 slice 被排定的時刻觸發 queued 路徑，該路徑直接設定重排，不重新套用嚴格不等號。

6.7　tick、HRTICK 與 sleep placement 的敏感度 （續）

週期 tick 的第二個條件是：delta_exec≥minimum_granularity 後，若 current 的 vruntime 大於 leftmost 且差值超過 ideal_runtime，也可重排。這個條件處理錯過喚醒邊界的工作，不能取代喚醒比較。

last/next/skip buddy 是 cache locality 與避免過度排程的候選提示，不是另一個無視 vruntime 的 priority。此例在 P3 尚睡時 n<8，未形成一般 next buddy；真正 wakeup 超過 granularity 時才把 next buddy 指向 P3。若 buddy 與 leftmost 的 virtual-time 差距太大，仍不能越過公平界線。

6.7　nice、weight 與跨案例比較

Linux 3.2 的 nice 不直接加減 priority queue；它改變 weight。v3.2 的 nice=0 weight 為 1024，nice=5 weight 為 335。若只有這兩個 task runnable 且 period=6 ms：

task	nice	weight	ideal_runtime	實體每 1 ms 增加的 vruntime
A	0	1024	6×1024/1359≈4.521 ms	1 ms
B	5	335	6×335/1359≈1.479 ms	1024/335≈3.057 ms

所以 nice 低者取得約 75.35% 的 CPU，nice 高者取得約 24.65%；權重並非線性 CPU percentage。n>8 時 period 隨 runnable 數延長，讓 minimum granularity 避免 slice 無限變短。

案例	固定版本規則	同一 logical workload 的 completion P1/P2/P3
Solaris TS/IA	p20/q120 ms；sleep 返回提升到 p50	8/14/11
Windows 7	32 級 queue；normalized 2/6 tick；I/O boost	14/8/11
Linux 3.2 CFS	HRTICK；nice=0；6 ms period；P3 t=9 wake	14/12/11

6.7　nice、weight 與跨案例比較 （續）

這張表比較的是事先明定的事件模型，不是三個產品的 benchmark。若 Linux 關閉 HRTICK，上述 completion 改為 14/8/11；若 P3 在 t=8.5 醒來，則為 14/11.5/10.5。可見「同一 workload」仍不足以推出唯一 Gantt，tick 模式與事件時刻必須先固定。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

6.8　演算法評估

- deterministic modeling 以代表性時間表精算指標；queueing model 以 arrival 與 service 分布估計平均；simulation 能加入更複雜政策；implementation 量測才有真實硬體成本。
- 模型必須揭露 workload 分布、CPU 數、context switch 成本與追蹤窗口。只報平均值容易隱藏 starvation 與尾端延遲。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

6.8　模型、時序與反例：演算法評估

模型假設。 模型輸入分布、arrival、service、CPU 數與 switch cost 一致；simulation 的 seed 與置信區間保留。

不變量與時序。 不同方法以同一簡化 workload 與成本假設做對照；若結論不同，要查明被省略的成本或分布差異，不能把一致方向當演算法不變量。比較結果保留相同量測窗口。

反例或邊界。 deterministic 例子顯示 SJF 較好，就直接推論所有 burst estimate 誤差下的實際系統也較好。

完整算例。 改以可完整重算的deterministic trace比較：單CPU、P1/P2/P3 arrival皆0、burst=5/2/3ms，無I/O與切換成本。Response固定為首次取得CPU；p99使用nearest-rank，三筆資料取第ceil(0.99×3)=3筆，即本批最大值。Waiting=completion−burst。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

6.8 模型、時序與反例：演算法評估 (續)

政策	completion P1/P2/P3	waiting P1/P2/P3	平均 waiting	first response P1/P2/P3	本批p99 response
FCFS，初序 P1/P2/P3	5/7/10	0/5/7	4	0/5/7	7
SJF，P2/P3/P1	10/2/5	5/0/2	7/3	5/0/2	5
RR q=2，初序 P1/P2/P3	10/4/9	5/2/6	13/3	0/2/4	4

三者觀察窗口皆[0,10]，throughput=3/10 jobs/ms；SJF平均等待較小，RR本批首CPU尾端較小，不能用一個平均值替代另一指標。這是三個給定jobs的精算，不是隨機模擬或母體p99估計，沒有可憑空宣稱的95%置信區間。若改做隨機實驗，須另給arrival/service分布、seed、獨立重複單位與估計方法；平均值的置信區間也不是涵蓋95%個別等待的範圍。

思考題。 如何讓 benchmark 不只看平均吞吐而看見 tail？

6.9 總結

- 排程是 policy 問題。正確性包括所有 state 的 queue ownership 與 no double-run；服務品質還要看等待上限與回應分布。
- CPU utilization 高不保證使用者等待短；在共享資源競爭下，更多 runnable 工作反而可能降低有效吞吐。

6.9　模型、時序與反例：總結

模型假設。 排程正確性、服務品質與成本模型都被分開驗證。

不變量與時序。 若服務契約要求 starvation freedom，須在 admission 與 arrival 假設下保證 runnable thread 最終前進；fixed-priority 本身可能不提供此性質，不能把所有政策都宣稱公平。

反例或邊界。 CPU 利用率 99% 的 batch 政策，若互動工作被永久延後，仍不符合互動系統目標。

完整算例。 沿用RR q=2、arrival全0、初序P1/P2/P3，只把三個bursts同乘k。每個時間片從ready queue取head，跑min(2,remaining)，未完成才入tail。逐段重播所得：

k；bursts	completion P1/P2/P3	waiting P1/P2/P3	throughput jobs/ms	本批p99首CPUresponse	最大waiting
1；5/2/3	10/4/9	5/2/6	3/10	4	6
2；10/4/6	20/10/16	10/6/10	3/20	4	10
3；15/6/9	30/16/25	15/10/16	1/10	4	16

6.9　模型、時序與反例：總結 (續)

例如k=2的執行順序為P1[0,2)、P2[2,4)、P3[4,6)、P1[6,8)、P2[8,10)、P3[10,12)、P1[12,14)、P3[14,16)、P1[16,18)、P1[18,20)。三列first response都0/2/4，busy utilization皆100%，最大ready等待卻6→10→16；所以首CPUresponse、總吞吐與等待必須一起看。這個有限batch不證明無限arrival下沒有starvation。

思考題。 哪個公平指標能補足平均利用率看不到的 starvation？

本章收束

- 演算法名稱不能脫離 burst 與 arrival 假設
- 回應與周轉時間不可混用
- 多處理器與即時排程要先界定 locality、期限與失敗模型

條件檢查。 使用本章任何模型前，先問它的假設是否仍成立。尤其是排程負載、同步保證、記憶體引用模式與硬體錯誤模型。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

練習與參考資料

練習題與詳解分別見本章獨立 PDF。

- Operating System Concepts, 9th Edition，第 6 章，印刷頁 261-312。
- *Operating System Concepts*, 9th Edition, Section 6.7, pp. 290-299：Linux CFS、Windows 7 與 Solaris class/priority/quantum 的版本案例。
- Linux v3.2 `kernel/sched_fair.c`：`__sched_period` 與 `sched_slice`、`place_entity`、`check_preempt_tick`、HRTICK 與 wakeup preemption。核心行段見 495-543、952-1015、1119-1153、1862-1905 與 2520-2662。
- Linux v3.2 `kernel/sched.c`：nice −20 至 19 的 `prio_to_weight`；nice=0 為 1024、nice=5 為 335。
- Microsoft Learn: Priority Boosts 與 Scheduling Priorities：Windows priority/quantum 與 foreground/I/O boost。
- Oracle Solaris Time-Sharing Class：time-sharing priority/quantum 與 sleep/CPU-use 調整。
- Windows normalized ticks 與跨 OS logical units 只供同工作量的選取比較，不構成產品效能量測。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

練習與參考資料 （續）

- 教材把 P1/P2 初始 vruntime 正規化為 0；這避免把任務建立時的 `START_DEBIT` 絕對 offset誤當成本例的相對差異。HRTICK、週期 tick、喚醒時刻與 tie-break 都是模型輸入，不由 Gantt 反推。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Abraham Silberschatz, Peter B. Galvin, Greg Gagne. Operating System Concepts, 9th Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。
https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study